Figure 2. Learning curves for the closed environments. Mean of three random seeds, shading from min to max.

the same $(\text{op}; \text{term})$ actions deterministically reproduces the solution. We exploit this by enabling a light-weight success replay path in the env, where the solution traces $(s_t; a_t)_t$ are stored to a memory buffer and periodically mixed into the on-policy rollouts.

Relabel constants. To improve parameter sharing across structurally identical equations that differ only by symbol names, we introduce a relabel-constants action. When invoked, it applies a consistent partial renaming (e.g., $\{d \mapsto a;\ e \mapsto b;\ ::: \}$) to the current main equation and the working state, mapping arbitrary coefficients onto a compact canonical alphabet $\{a; b; c\}$. The environment stores the induced substitution map on a stack (`map_constants_history`) and uses the mapped equation for all subsequent steps. Upon solving, we unwind the stack in reverse, substituting back the original symbols so the final answer is expressed in the user's variables. This reduces input vocabulary entropy, prevents spurious overfitting to symbol IDs, and makes action terms like "subtract b" reusable across many instances.

Curiosity based rewards. In previous work (?), curiosity-based exploration bonuses (e.g. ICM, RND, NGU) were shown to enhance learning on a regular MLP policy. In preliminary experiments we found these bonuses did not improve, and in several cases degraded, performance when stacked on top of the TreeMLP; we therefore do not use them in our headline results (details deferred to a future revision).

Table 1. Greedy test accuracy for each agent across dataset. Single-seed results; ConPoLe / ConPoLe-local numbers from (?). "–" denotes a cell awaiting a multi-seed re-run.

Algorithm	small	large	poesia
ConPoLe-local	n/a	n/a	0.765
ConPoLe	–	–	0.925
ppo	0.04	0.02	0.17
ppo-tree	–	–	0.815
ppo-tree-rc	0.90	–	0.96
ppo-tree-rc-buf	0.80	0.44	–

3. Results: Closed equations

Algorithms. We use the standardized implementation of ppo from stable-baselines 3. We experimented with A2C and DQN but found worse performance. Hyperparameters, chosen via tuning, are reported in the Appendix. We trained for 10^7 steps and evaluated every 5×10^5. We used three random seeds.

Datasets. We tasked the agent with solving a random equation during each episode. We generate train/test sets by starting with x and applying actions recursively. Recall actions are $(operation; term)$ pairs. The operations are as before, but we restrict terms to $(a; b; c)$ (we excluded $d; e$ since we wanted to favor long, rather than wide equations). Applying a single action generated equations like $ax; \log(x); x+b$, applying 2 actions equations like $ax+b; \log(x)+d; (x+b)$=c and so on. We threw out any equations that SymPy couldn't solve, and considered multi-root equations solved when just

chine Learning (ICML 2000), pp. 1207–1216, Stanford, CA, 2000.

Ng, A. Y., Coates, A., Diel, M., Ganapathi, V., Schulte, J., Tse, B., Berger, E., and Liang, E. Autonomous inverted helicopter flight via reinforcement learning. In Experimental Robotics IX, pp. 363–372. Springer, 2006.

Poesia, G., Dong, W., and Goodman, N. Contrastive reinforcement learning of symbolic reasoning domains. In Advances in Neural Information Processing Systems, volume 34, pp. 17729–17741, 2021.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347, 2017.

Vinyals, O., Babuschkin, I., Czarnecki, W. M., Mathieu, M., Dudzik, A., Chung, J., Choi, D. H., Powell, R., Ewalds, T., Georgiev, P., et al. Grandmaster level in StarCraft II using multi-agent reinforcement learning. Nature, 575 (7782):350–354, 2019.

.1. Macroactions

Here we justify why the three macroactions $(\mathrm{expand}; \text{None})$, $(\mathrm{collect}; x)$, $(\mathrm{mul}; -1)$ discussed in the main text are required to solve the equations we consider, not mere conveniences for the RL agent. Consider the action set without these macroactions:

$$O = \{\mathrm{add}; \mathrm{sub}; \mathrm{mul}; \mathrm{div}\} \cup \{\mathrm{square}; \mathrm{sqrt}; \mathrm{exp}; \mathrm{log}; \mathrm{sin}; \mathrm{cos}; \mathrm{asin}; \mathrm{acos}\}; \tag{11}$$

$$T = \mathrm{SubExpr}(\mathrm{lhs}) \cup \mathrm{SubExpr}(\mathrm{rhs}); \tag{12}$$

$$A = (O \times T): \tag{13}$$

Now consider the equation $dx + c(ax + b) = e$. The agent must distribute c into the bracketed term $ax + b$, which the action set above cannot do: expand is required. Once expanded to $dx + cax + cb = e$, one needs to factor the x common to the first two terms—this is what $(\mathrm{collect}; x)$ provides. Finally, consider $-x = b$: since -1 is not in our term set, $(\mathrm{mul}; -1)$ is needed to isolate x. (Alternatively, -1 could be added to the term set, but we chose to keep the term set purely symbolic.)

.2. Dataset generation: Rational Equation

To supplement the recursive datasets, we constructed rational equations of the form

$$\frac{ax + b}{cx + a} + b = 0;$$

where $a; b; c$ are (non-zero) symbolic coefficients. We excluded degenerate cases such as denominators that vanish identically or cancel trivially with numerators.

Examples of retained equations include:

$$\frac{x + b}{cx + a} = 0 \quad \Rightarrow \quad x = -b;$$

$$\frac{ax + b}{cx + a} + b = 0 \quad \Rightarrow \quad x = \frac{-b - ab}{a + cb};$$

$$\frac{ax + b}{cx + a} - c = 0 \quad \Rightarrow \quad x = \frac{-b + ac}{a - c^2}:$$

These rational forms were merged into both the small and large datasets, with their proportion limited to preserve diversity across other functional forms (logarithmic, trigonometric, exponential, etc.).

.3. Illegal actions

The main issue was to avoid illegal division by zero. This is not as simple as precluding $(\mathrm{div}; 0)$ from the action set. We also had to prohibit hidden division by zero—for example, dividing by $(x + a)$ or $(x + b)$ when the equation has the form $(x + a)(x + b) = 0$, since one of the factors will vanish at the solution. To handle this, we check whether the current equation factors as $P(x)Q(x) \cdots = 0$ with $P; Q; :::$ polynomial in x, and remove $(\mathrm{div}; P); (\mathrm{div}; Q); :::$ from the action set for that state.

.4. Hyperparameters

We used Stable Baselines 3's default hyperparameters for A2C and PPO (Table ??).

Algorithm	Hyperparameters
PPO	$learning\_rate = 0{:}0003$, $n\_steps = 2048$, $batch\_size = 64$, $n\_epochs = 10$, $gamma = 0{:}99$, $gae\_lambda = 0{:}95$, $clip\_range = 0{:}2$, $ent\_coef = 0{:}0$, $vf\_coef = 0{:}5$, $max\_grad\_norm = 0{:}5$, $normalize\_advantage = True$, $device = ' auto'$
PPO-tree	$learning\_rate = 0{:}0007$, $n\_steps = 5$, $gamma = 0{:}99$

Table 3. Default hyperparameters for A2C and PPO from Stable Baselines 3.

.5. Solution Traces

Below are the solution traces for each equation in the fixed equation environment. Notice the solution trace is not always optimal. For instance, $ax + b = 0$ could be solved with $(sub; b); (div; a)$, but instead the agent selects $(sub; ax)$ and then $(mul; -1)$. "Truediv" is the sympy notation for divide.

1. Equation: $a + x = 0$

```
Step 1: a + x = 0 | subtract, a
Solved: x = -a
```

2. Equation: $ax = 0$

```
Step 1: a*x = 0 | div, a
Solved: x = 0
```

3. Equation: $ax + b = 0$

```
Step 1: ax + b = 0 | subtract, ax
Step 2: -ax = b | multiply, -1
Step 3: ax = -b | div, a
Solved: x = -b/a
```

4. Equation: $a=x + b = 0$

```
Step 1: a/x + b = 0 | subtract, b
Step 2: a/x = -b | truediv, 1/x
Step 3: -b*x = a | truediv, b
Step 4: -x = a/b | multiply, -1
Solved: x = -a/b
```

5. Equation: $c(ax + b) + d = 0$

```
Step 1: c*(ax + b) + d = 0 | expand, None
Step 2: acx + bc + d = 0 | subtract, acx
Step 3: -acx = bc + d | multiply, -1
Step 4: acx = -bc - d | truediv, c
Step 5: ax = (-bc - d)/c | truediv, a
Solved: x = (-bc - d)/(ac)
```

6. Equation: $c + d=(ax + b) = 0$

```
Step 1: c + d/(ax + b) = 0 | subtract, c
Step 2: d/(ax + b) = -c | multiply, (ax + b)
Step 3: d = -c (ax + b) | expand, None
Step 4: d = -c ax - c b | multiply, -1
Step 5: -d = c ax + c b | subtract, c b
Step 6: -d - c b = c ax | truediv, c
Step 7: (-d - c b)/c = ax | truediv, a
Step 8: ((-d - c b)/c)/a = x | truediv, 1/a
Solved: x = (-d - c b)/(c a)
```

7. Equation: $cx + d + e(ax + b) = 0$

Algorithm 1 TreeMLP message passing and pooling

Require: node IDs X, edges $(\mathrm{src}; \mathrm{dst})$, masks $m_{\text{node}}; m_{\text{edge}}$, steps K

$\mathrm{emb} \leftarrow \mathrm{Proj}(\mathrm{Embed}(\mathsf{X}))$ ▷ shape $[B; N; H]$
$h \leftarrow \mathrm{emb}$
for $k = 1$ to K **do**
  $\mathcal{E} \leftarrow \{(i \to j) \mid m_{\text{edge}}(i \to j) = 1\}$
  $\mathrm{agg}[j] \leftarrow \sum_{(i \to j) \in \mathcal{E}} h[i]$ ▷ scatter-sum
  $h \leftarrow \mathrm{MLP}\ [\mathrm{emb}; \mathrm{agg}]$
  $h \leftarrow h \odot m_{\text{node}}$ ▷ mask padded nodes
end for
return $\mathrm{pool}(h; m_{\text{node}})$ ▷ mean or max

```
Step 1: cx + d + e(ax + b) = 0 | expand, None
Step 2: aex + be + cx + d = 0 | collect, x
Step 3: be + d + x*(ae + c) = 0 | subtract, x(ae + c)
Step 4: -x(ae + c) = be + d | truediv, ae + c
Step 5: -x = (be + d)/(ae + c) | multiply, -1
Solved: x = -(be + d)/(a*e + c)
```

8. Equation: $e + (ax + b) = (cx + d) = 0$

```
Step 1: e + (ax + b)/(cx + d) = 0 | truediv, 1/(cx + d)
Step 2: (e + (ax + b)/(cx + d))(cx + d) = 0 | expand, None
Step 3: ax + b + cex + de = 0 | collect, x
Step 4: b + de + x*(a + ce) = 0 | subtract, x(a + ce)
Step 5: -x(a + ce) = b + de | expand, None
Step 6: -ax - cex = b + de | collect, x
Step 7: x*(-a - ce) = b + de | truediv, -a - ce
Solved: x = (b + de)/(-a - c*e)
```

.6. TreeMLP Details

Architecture. TreeMLP embeds node IDs, projects to a hidden size, and performs K message-passing stages over the (directed) expression tree. At each stage it sums incoming neighbor states, concatenates the result with the fixed input embedding, and applies a two-layer MLP with ReLU. A masked global pool (mean or max) produces the graph embedding exposed to the policy/value heads.

Listing 1. TreeMLP forward pass (minimal).

```
def forward(self, obs: dict) -> torch.Tensor:
    node_ids  = self._to_device(obs["node_features"], self.device)
    edge_index= self._to_device(obs["edge_index"], self.device)
    node_mask = self._to_device(obs["node_mask"], self.device)
    edge_mask = self._to_device(obs["edge_mask"], self.device)

    node_idx = self._id_to_idx(node_ids) if node_ids.dtype != torch.long else node_ids
    B, N = node_idx.shape
    _, _, E = edge_index.shape

    emb = self.embed(node_idx)                  # [B,N,embed_dim]
    emb = self.embed_proj(emb)                  # [B,N,H]
    h = emb.clone()

    src = edge_index[:, 0, :].long()           # [B,E]
    dst = edge_index[:, 1, :].long()           # [B,E]
    batch_idx = torch.arange(B, device=self.device).unsqueeze(1).expand(B, E)
```

```
    for _ in range(self.K):
        src_flat = src.reshape(-1)
        dst_flat = dst.reshape(-1)
        b_flat   = batch_idx.reshape(-1)
        m_flat   = edge_mask.reshape(-1).bool()

        src_flat, dst_flat, b_flat = src_flat[m_flat], dst_flat[m_flat], b_flat[m_flat]
        src_idx = b_flat * N + src_flat
        dst_idx = b_flat * N + dst_flat

        h_flat   = h.reshape(B * N, self.hidden_dim)
        messages = h_flat[src_idx]                          # [M,H]

        agg_flat = torch.zeros_like(h_flat)                 # [B*N,H]
        agg_flat.index_add_(0, dst_idx, messages)           # sum incoming
        agg = agg_flat.reshape(B, N, self.hidden_dim)       # [B,N,H]

        h = self.node_mlp(torch.cat([emb, agg], dim=-1))    # [B,N,H]
        h = h * node_mask.unsqueeze(-1).float()

    if self.pooling == "mean":
        denom = node_mask.sum(dim=1, keepdim=True).clamp(min=1).float()
        g = h.sum(dim=1) / denom
    else:
        if self._minus_inf is None or self._dtype_cache != h.dtype:
            self._minus_inf = torch.finfo(h.dtype).min
            self._dtype_cache = h.dtype
        g = h.masked_fill(~node_mask.unsqueeze(-1), self._minus_inf).max(dim=1).values

    return self.proj(g)
```

Implementation notes. We use fixed-size padding with boolean masks for nodes/edges, scatter-add for neighbor aggregation, and pool with mask awareness. All tensors are moved to CUDA when available; no MPS is used.

.7. Ablations

We summarize the ablation findings; the corresponding learning curves are in Figures ?? and ??.

- Replacing dense complexity-delta rewards with sparse outcome-only rewards (and disabling the inverse-frequency curriculum) substantially degrades performance. The main drawback of sparse rewards is far longer run times: we had to introduce a 1-second-per-step wall-clock timeout to keep training tractable.
- TreeMLP outperforms a vanilla GCN and GraphSAGE on the small dataset (Figure ??); the tree-aware aggregation appears necessary, not just helpful.
- In preliminary runs, adding curiosity bonuses (ICM, RND, NGU, RIDE, RE3, E3B) on top of TreeMLP did not yield consistent gains and in several cases degraded performance. This contrasts with earlier work (?), where curiosity improved a vanilla MLP policy. A possible explanation is that the TreeMLP's structural inductive bias already supplies the exploration signal curiosity provides for the MLP, making the two redundant or actively conflicting. We leave a full sweep across curiosity methods $\times$ seeds for the camera-ready version.

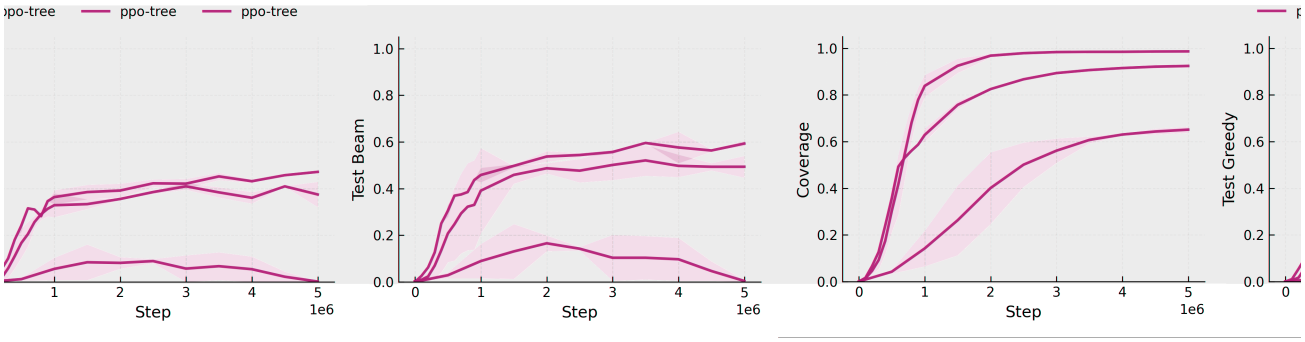

Figure 5. TreeMLP outperforms GCN and GraphSAGE on the small datasets. Mean of three random seeds with shading to min/max.

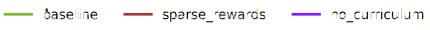

Figure 6. Ablation study on small datasets.